МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ РОССИЙСКОЙ ФЕДЕРАЦИИ

Федеральное государственное бюджетное образовательное учреждение высшего образования

«Казанский национальный исследовательский технический университет им. А.Н. Туполева – КАИ»

Институт компьютерных технологий и защиты информации

Кафедра Систем Информационной Безопасности

Лабораторная работа № 2

Дисциплина: «Аппаратные средства телекоммуникационных систем»

Тема: «Параллельное перемножение больших матриц»

Работу выполнил

Студент гр. 4404

Ионов И.А.

Казань, 2026

1. Цель работы

Параллелизовать перемножение матриц и сравнить время выполнения последовательной и параллельной версий на 2, 4 и 8 потоках.

2. Постановка задач

Задача 4. Написать программу перемножения больших матриц. Сравнить время выполнения последовательной и параллельной программы на 2, 4 и 8 потоках (процессорах). Готовое умножение матриц не использовать.

3. Реализация

Файл lab2/Task4.java. Матрицы A и B заполняются случайными числами (фиксированный seed). Умножение реализовано тройным циклом по i, j, k; матрица B предварительно транспонируется, чтобы внутренний цикл по k шёл по строкам обеих матриц (последовательный доступ к памяти).

В параллельной версии строки результирующей матрицы C делятся на непрерывные блоки, по одному на поток (аналог schedule(static) у внешнего цикла). Запись идёт в разные строки C, поэтому зависимости по данным между потоками нет и синхронизация не нужна.

Перед измерением выполняется прогрев JIT-компилятора; для каждой версии берётся лучшее время из нескольких запусков. Результат параллельной версии сравнивается с последовательным, выводятся ускорение и эффективность.

Сборка и запуск (подробнее в README папки lab2):

```
cd lab2
javac Task4.java
java Task4 1000 2,4,8 3
```

4. Ход работы и скриншоты

Рабочий код:

```
import java.util.Random;

/**
 * Задача 4. Перемножение больших матриц C = A * B, сравнение последовательной и параллельной версий.
 * Параллельная версия: строки результата делятся на непрерывные блоки (аналог schedule(static)),
 * каждый поток считает свой блок строк. Готовые функции умножения матриц не используются.
 *
 * Запуск: java Task4 [размер=800] [потоки=2,4,8] [повторов=3]
 *   потоки - список через запятую, например 2,4,8
 */
public class Task4 {

    /** Умножение строк [from, to) результата. B транспонирована -> оба внутренних обращения идут по строкам. */
    static void mulRows(double[][] a, double[][] bt, double[][] c, int n, int from, int to) {
        for (int i = from; i < to; i++) {
            double[] ai = a[i];
            double[] ci = c[i];
            for (int j = 0; j < n; j++) {
                double[] bj = bt[j];
                double s = 0;
                for (int k = 0; k < n; k++) s += ai[k] * bj[k];
                ci[j] = s;
            }
        }
    }

    static double[][] transpose(double[][] m, int n) {
        double[][] t = new double[n][n];
        for (int i = 0; i < n; i++)
            for (int j = 0; j < n; j++) t[j][i] = m[i][j];
        return t;
    }

    static double[][] multiplySeq(double[][] a, double[][] b, int n) {
        double[][] c = new double[n][n];
        mulRows(a, transpose(b, n), c, n, 0, n);
        return c;
    }

    static double[][] multiplyPar(double[][] a, double[][] b, int n, int p) throws InterruptedException {
        double[][] c = new double[n][n];
        double[][] bt = transpose(b, n);
        Thread[] ts = new Thread[p];
```

```
        int base = n / p, rem = n % p;
        int from = 0;
        for (int t = 0; t < p; t++) {
            int to = from + base + (t < rem ? 1 : 0);
            final int f = from, e = to;
            ts[t] = new Thread(() -> mulRows(a, bt, c, n, f, e));
            ts[t].start();
            from = to;
        }
        for (Thread t : ts) t.join();
        return c;
    }

    static double maxDiff(double[][] x, double[][] y, int n) {
        double d = 0;
        for (int i = 0; i < n; i++)
            for (int j = 0; j < n; j++) d = Math.max(d, Math.abs(x[i][j] - y[i][j]));
        return d;
    }

    public static void main(String[] args) throws InterruptedException {
        int n = args.length > 0 ? Integer.parseInt(args[0]) : 800;
        String[] plist = (args.length > 1 ? args[1] : "2,4,8").split(",");
        int reps = args.length > 2 ? Integer.parseInt(args[2]) : 3;
        if (n < 1 || reps < 1) { System.err.println("размер >= 1, повторов >= 1"); return; }

        Random rnd = new Random(42);
        double[][] a = new double[n][n], b = new double[n][n];
        for (int i = 0; i < n; i++)
            for (int j = 0; j < n; j++) { a[i][j] = rnd.nextDouble(); b[i][j] = rnd.nextDouble(); }

        System.out.printf("Matrix size: %d x %d, repetitions: %d, available cores: %d%n",
                n, n, reps, Runtime.getRuntime().availableProcessors());

        // прогрев JIT-компилятора на небольшой матрице
        int w = Math.min(n, 150);
        double[][] wa = new double[w][w], wb = new double[w][w];
        for (int i = 0; i < w; i++) for (int j = 0; j < w; j++) { wa[i][j] = i + j; wb[i][j] = i - j; }
        for (int r = 0; r < 5; r++) { multiplySeq(wa, wb, w); multiplyPar(wa, wb, w, 2); }

        double bestSeq = Double.MAX_VALUE;
        double[][] ref = null;
        for (int r = 0; r < reps; r++) {
            long t0 = System.nanoTime();
            ref = multiplySeq(a, b, n);
            bestSeq = Math.min(bestSeq, (System.nanoTime() - t0) / 1e9);
        }
        System.out.printf("%-12s %10s %10s %12s %s%n", "version", "time, s", "speedup",
"efficiency", "check");
        System.out.printf("%-12s %10.3f %10s %12s %s%n", "sequential", bestSeq, "1.00", "-", "-");

        for (String ps : plist) {
```

```
            int p = Integer.parseInt(ps.trim());
            double best = Double.MAX_VALUE;
            double[][] c = null;
            for (int r = 0; r < reps; r++) {
                long t0 = System.nanoTime();
                c = multiplyPar(a, b, n, p);
                best = Math.min(best, (System.nanoTime() - t0) / 1e9);
            }
            double speedup = bestSeq / best;
            boolean ok = maxDiff(ref, c, n) == 0.0;
            System.out.printf("%-12s %10.3f %10.2f %11.0f%% %s%n", p + " threads", best, speedup,
100.0 * speedup / p, ok ? "OK" : "MISMATCH");
        }
        System.out.println("(time = best of " + reps + " runs)");
    }
}
```

Скриншот 1. Запуск: `java Task4 500 2,4,8`

```
Matrix size: 500 x 500, repetitions: 3, available cores: 12
version         time, s    speedup   efficiency check
sequential        0,069       1.00            - -
2 threads         0,038       1,84          92% OK
4 threads         0,029       2,39          60% OK
8 threads         0,023       2,94          37% OK
(time = best of 3 runs)
```

Пояснение. Матрицы 500 × 500: время последовательной версии и версий на 2, 4, 8 потоках, ускорение, эффективность и проверка совпадения результатов (OK).

Скриншот 2. Запуск: `java Task4 1000 2,4,8`

```
Matrix size: 1000 x 1000, repetitions: 3, available cores: 12
version         time, s    speedup   efficiency check
sequential        0,525       1.00            - -
2 threads         0,287       1,83          91% OK
4 threads         0,174       3,01          75% OK
8 threads         0,139       3,77          47% OK
(time = best of 3 runs)
```

Пояснение. Матрицы 1000 × 1000. С ростом размера доля накладных расходов на создание потоков уменьшается, и ускорение приближается к числу доступных ядер.

Скриншот 3. Запуск: `java Task4 1500 2,4,8`

```
Matrix size: 1500 x 1500, repetitions: 3, available cores: 12
version         time, s    speedup   efficiency check
sequential        2,088       1.00            - -
2 threads         1,153       1,81          91% OK
4 threads         0,673       3,10          78% OK
8 threads         0,645       3,24          40% OK
(time = best of 3 runs)
```

Пояснение. Матрицы 1500 × 1500: объём вычислений растёт как n^3, выигрыш от распараллеливания максимален.

5. Ответы на вопросы

Вопрос. Как меняется время выполнения при увеличении числа потоков?

Ответ. Время уменьшается, пока число потоков не превышает число физических ядер процессора. Ускорение S = T(посл.) / T(парал.) для p потоков не превышает p, эффективность E = S / p обычно падает с ростом p. Если потоков больше, чем ядер (например, 8 потоков на 4 ядрах), дальнейшего выигрыша нет: потоки конкурируют за ядра, а накладные расходы на их создание и переключение сохраняются. На малых матрицах выигрыш мал или отсутствует, так как стоимость создания потоков сопоставима с временем самого счёта.

Вопрос. Почему параллельное умножение корректно без синхронизации?

Ответ. Каждый поток записывает элементы только своих строк матрицы C, а матрицы A и B только читает. Итерации внешнего цикла независимы, то есть зависимости по данным нет (в отличие от варианта, где внешним был бы цикл по k и все потоки записывали бы в одни и те же a[i][j]).

6. Выводы

- Умножение матриц хорошо распараллеливается: итерации внешнего цикла независимы, синхронизация не нужна.
- Ускорение ограничено числом физических ядер; при росте размера матриц ускорение растёт, так как накладные расходы становятся малы по сравнению с объёмом вычислений.
- Корректность параллельной версии подтверждена совпадением с результатом последовательной версии.